

Table of Contents

MODEL SELECTION	2
Occam's Razor	2
Overfitting.....	4
Bias-Variance Trade-Off	5
Regularisation.....	6
Models, Algorithms and Hypothesis Classes	7
Metrics and Business Objectives.....	8
Validation Sets and Generalisation	8
Hyperparameters.....	10
Hyperparameter Search Techniques	10
Common Pitfalls in Cross-Validation	11
Curse of Dimensionality and Feature Selection	12
Feature Selection Approaches	12
Conclusion	13

Model Selection

The central issue in machine learning is the extrapolation of the learning from a finite amount of data to all possible inputs of the same type. We build models from some training data, but the training data is always finite.

On the other hand, a model is expected to learn enough about the entire domain from which the data is drawn. In almost all realistic scenarios, the domain is infinitely large. How can we ensure that a model that performs well on training data will also perform well on unseen data?

Model selection shifts the emphasis from fitting models to choosing which model, configuration and feature set should be trusted for deployment. These choices are guided by domain understanding and validated using systematic evaluation strategies.

A model represents a hypothesis about how input variables generate outcomes. Different model families make different assumptions about this relationship. For example, linear regression assumes a linear relationship between inputs and output, whereas decision trees assume the target can be explained through hierarchical splits of the feature space. Choosing a model is equivalent to choosing a hypothesis class that is suitable for the problem.

There is no single best model for all problems. The model choice depends on the data, business objective, interpretability requirements and computational constraints.

Occam's Razor

A predictive model should be as simple as possible, without compromising accuracy. It is often called Occam's Razor. This is not just a matter of convenience but a fundamental rule of thumb in machine learning.

Before exploring this further, let's get an intuitive understanding of what it means for a model to be simple. To measure the simplicity, we often use its complementary notion, that of the complexity of a model.

There is no universal definition of the complexity of a model used in machine learning (ML).

Here are a few typical ways of assessing a model's complexity:

1. Number of parameters required to specify the model completely

For example, in simple linear regression, for the response attribute y on the explanatory attributes x_1, x_2, x_3 , the model $y = ax_1 + bx_2$ is simpler than the model $y = ax_1 + bx_2 + cx_3$, which requires three parameters compared with the two required for the first model.

2. The degree of the function if it is a polynomial

Considering regression, the model $y = ax_1^2 + bx_2^3$ would be a more complex one because it is a polynomial of degree 3.

3. Size of the best possible representation of the model

For instance, this could refer to the number of bits required in binary encoding of a model. The more complex (messy, requiring higher precision, involving large numbers, etc.) the coefficients in the model, the more complex the model is. For example, the expression $(0.552984567 * x^2 + 932.4710001276)$ is considered to be more 'complex' than $(2x + 3x^2 + 1)$, even though the second one has more terms in it.

4. The depth or size of a decision tree or the number of layers in a neural network

Intuitively, a complex model entails more assumptions. Therefore, Occam's Razor is a simple rule of thumb - given two models that show similar performance on finite training or test data, we should pick one that makes fewer assumptions about unseen data.

This means we need to pick the simpler of the two models. Generally, among the best performing models on the available data, we pick the one that makes the fewest assumptions, which would be the simplest among them. A deep relationship exists between a model's complexity and usefulness in a learning context.

Simpler models are usually more generic, more widely applicable and more robust. They are not sensitive to the specific noise of the training data but can generalise well to unseen data. Simpler models also require fewer training samples for effective training than more complex models (although the complex model might perform better during training).

An important goal of model selection is generalisation. A model trained in a controlled environment must be able to apply its learning to new and previously unseen situations. Validation performance, rather than training performance alone, provides insight into whether a model is learning meaningful structure or simply memorising the training data.

Overfitting

As model flexibility increases, training errors typically decrease, but validation errors may eventually increase. This behaviour reflects overfitting, where the model starts capturing noise rather than true relationships.

Overfitting is a phenomenon where a model becomes more complex than what is warranted for the task at hand and, consequently, suffers from bad generalisation properties.

A trivial model that would have learnt perfectly from any given data set is one that just memorises the entire data set. The error committed by such a model on the training data set would be zero.

Suppose we can try to fit a polynomial regression model on each point of the training data by increasing the degree to achieve an excellent training performance.

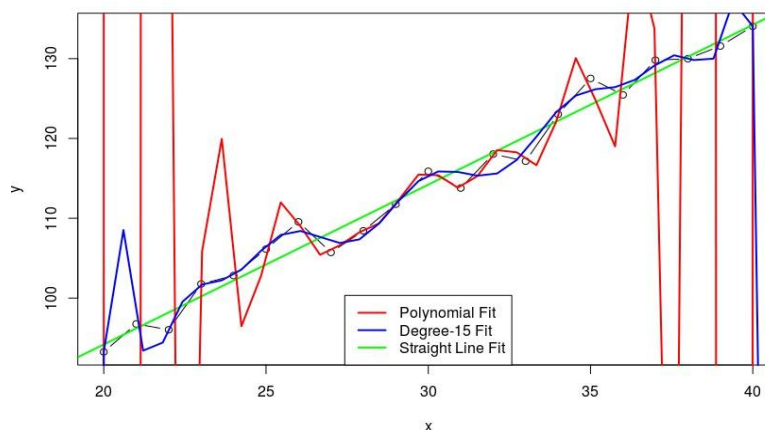


Figure 1: Polynomial vs Straight Line Fit

However, such a model can only answer questions (though perfectly) on the training data set. The model will effectively be incapable of performing better than a random guess on a test data point that is outside the training data set.

Bias-Variance Trade-Off

Suppose a model memorises the entire training data set. This model will need to change for even small changes in the data set. So, the model is highly unstable and extremely sensitive to changes in the training data. However, a simpler model that abstract patterns from the data is unlikely to change significantly even if points are added, removed or slightly perturbed.

Apart from random noise in data, prediction errors mainly arise from two sources: bias and variance.

Bias refers to errors caused by incorrect or overly simplistic assumptions. High-bias models are too rigid to capture the true data-generating process and tend to underfit, showing poor performance on training and test data.

Variance refers to errors caused by excessive sensitivity to the training data. High-variance models fit training data very closely, including noise, but perform poorly on new data owing to overfitting.

As model complexity increases, bias usually decreases while variance increases. Highly simple models suffer from high bias and low variance, whereas highly complex models experience low bias and high variance. Model selection aims to find an intermediate level of complexity, where both bias and variance are reasonably low.

This balance, called the bias–variance trade-off, explains why neither the simplest nor the most complex model is usually optimal. Instead, the best model is one that achieves stable performance across different data sets and generalises reliably beyond the training data. The figures shown below illustrate the same.

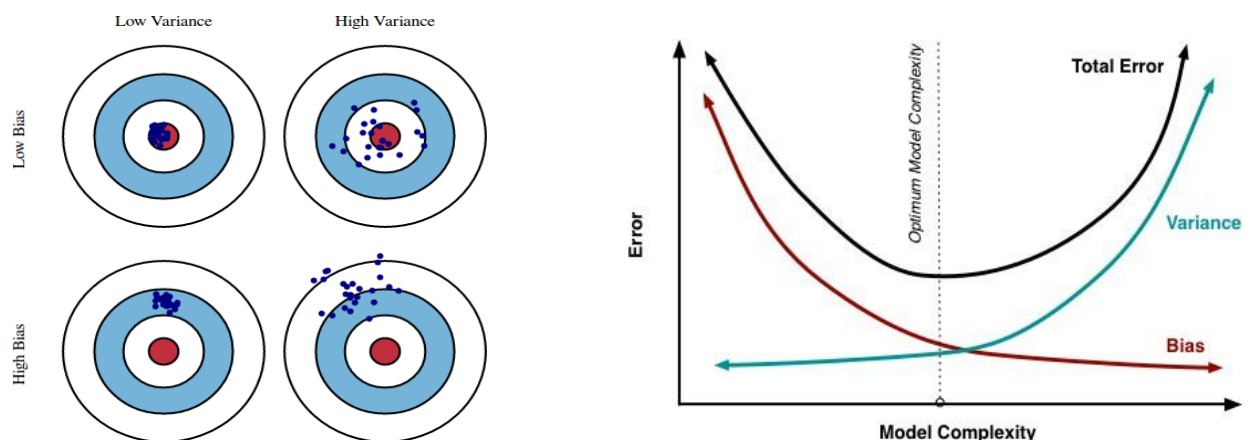


Figure 2:
(Left) Illustration of Bias and Variance (Right) Bias-Variance Trade-off

The left-hand side of Figure 2 illustrates bias and variance using a target shooting analogy. A model whose shots are clustered together has a small variance, and one whose shots are close to the bull's eye has a small bias.

A 'consistent' shooter will have a small variance, and a 'good' shooter will have a small bias. This means that variance is associated with consistency, and bias is related to accuracy.

The right-hand side of Figure 2 illustrates the typical trade-off between bias and variance. Low-complexity models have high bias and low variance, and high-complexity models have low bias but high variance.

Our naïve 'constant' model will never change; hence, its variance is zero although the bias is high. The most suitable model for a task is one that balances both, achieving a reasonable degree of predictability (low variance) without compromising a lot on the accuracy (bias).

Regularisation

Regularisation is used in machine learning to deliberately simplify a model. Through regularisation, we try to strike a delicate balance between keeping the model simple without making it too naïve to be useful, which leads to this question: How much error am I willing to tolerate during training to gain generalisability?



Figure 3: Regularisation

Simplification can occur at several levels — choosing simpler functions, keeping the number of model parameters small, keeping the degree of the polynomial low, etc. These are done at the model definition stage and are typically not called regularisation. Regularisation is a simplification applied during training to control model complexity.

Some typical regularisation steps for various classes of ML algorithms are as follows:

- For regression, this involves adding a regularisation term to the cost that adds the absolute values or the squares of the parameters of the model.
- For decision trees, this could mean 'pruning' the tree to control its depth and/or size.
- For neural networks, a common strategy is to include a dropout layer, removing a few neurons and/or weights at random.

Models, Algorithms and Hypothesis Classes

Distinguishing between a model and the learning algorithm used helps train the model. The model defines the form of the relationship between inputs and outputs, such as linear regression, decision trees or neural networks. The learning algorithm specifies how the parameters of that model are learnt from data, such as gradient descent, normal equations, CART or backpropagation.

A hypothesis is a specific model instance learnt by the algorithm, whereas a hypothesis class is the set of all possible hypotheses that the model family can express. A representational diagram is shown below.

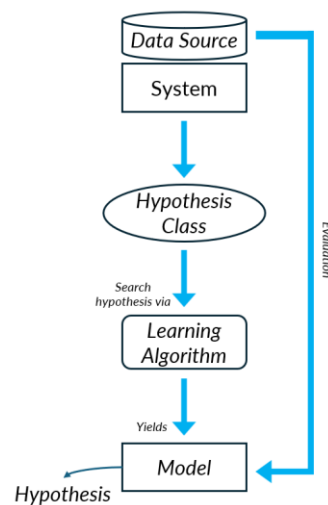


Figure 4: Hypothesis and Hypothesis Class

For example, linear regression corresponds to the class of all linear functions of the input variables. Model selection is fundamentally about choosing an appropriate hypothesis class, after which the learning algorithm searches that class to find a good hypothesis.

Metrics and Business Objectives

Model selection cannot be separated from the choice of evaluation metric. Different problems require different metrics, and the best-performing model according to only one metric may not be optimal for business objectives.

For example, in fraud detection, recall may be more important than accuracy, as missing fraudulent cases can be costly. In credit scoring, maximising the expected profit may matter more than classification accuracy. So, validation metrics should be aligned with business key performance indicators.

Validation results guide model selection towards outcomes that are significant in practice rather than abstract numerical improvements.

Validation Sets and Generalisation

Training accuracy alone is not a reliable indicator of real-world performance. To estimate how well a model will perform on unseen data, we evaluate it on data that was not used during training. A validation set is a held-out subset of the data used during model development to compare models and tune hyperparameters.

A common workflow is to split the data into training, validation and test sets. The training set is used to fit the model, the validation set is used for model tuning and selection, and the test set is reserved for final evaluation. Validation acts as an early warning system against overfitting and helps guide model selection decisions before deployment.

Holdout Validation and Cross-Validation

A simple approach to validation is the **holdout** method, where a single split is made between training and validation data. This approach is fast and works well for large data sets, but the results can vary depending on how the split is made. A diagram illustrating the same is shown below.

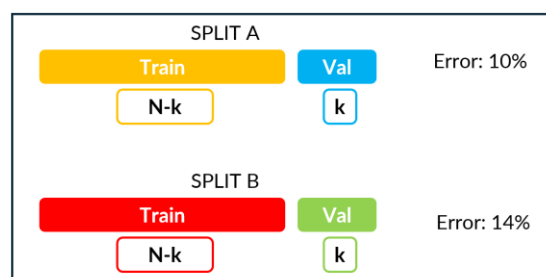


Figure 5: Holdout Validation

Cross-validation provides a more reliable estimate of generalisation performance. In cross-validation, the data is divided into multiple folds, and the model is trained and validated multiple times, each time using a different fold for validation. The final performance estimate is obtained by averaging the results across folds.

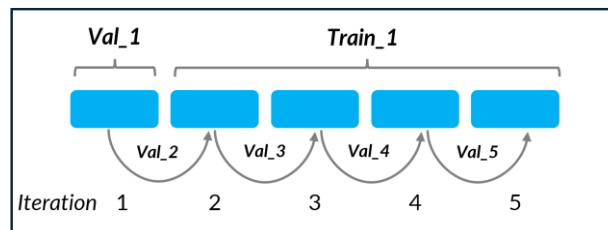


Figure 6: Cross-Validation

Cross-validation reduces variance in performance estimates and is particularly useful when data is limited. It is commonly implemented using utilities such as `KFold` or `cross_val_score` from scikit-learn.

Cross-Validation Strategies

In **k-fold cross-validation**, the data set is split into k folds. The model is trained on $k-1$ folds and validated on the remaining fold, repeating this process k times. Typical values of k are 5 or 10, balancing reliability and computational cost.

For classification problems with class imbalance, **stratified cross-validation** is preferred. `StratifiedKFold` ensures that each fold preserves the class proportions of the original data set, leading to more realistic performance estimates.

Leave-one-out cross-validation (LOOCV) is a special case where each data point acts as its own validation set. This approach has low bias but high variance and is computationally expensive, making it suitable mainly for very small data sets.

Time series data requires special handling because observations are temporally dependent. **Time series cross-validation** uses forward-chaining strategies, such as expanding or rolling windows, to ensure that training always precedes validation in time. This can be implemented using `TimeSeriesSplit`.

Hyperparameters

Hyperparameters are configuration values set before training that control how a model learns. They differ from model parameters, which are learnt from data. Examples of hyperparameters include tree depth, regularisation strength, number of layers in a neural network and learning rate.

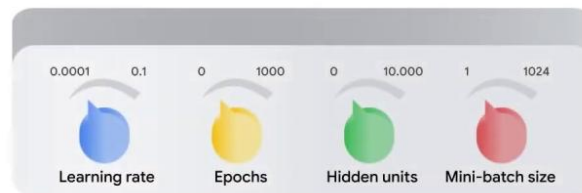


Figure 7: Hyperparameters in NN

Hyperparameter tuning aims to find a configuration that yields the best validation performance. This process must be combined with proper validation to ensure that observed improvements reflect true generalisation rather than chance.

Hyperparameter Search Techniques

Grid search evaluates all possible combinations of the specified hyperparameter values (as shown in the diagram below). It is systematic and exhaustive but computationally expensive as the search space grows. Grid search is commonly implemented using **GridSearchCV**, which integrates cross-validation into the search process.

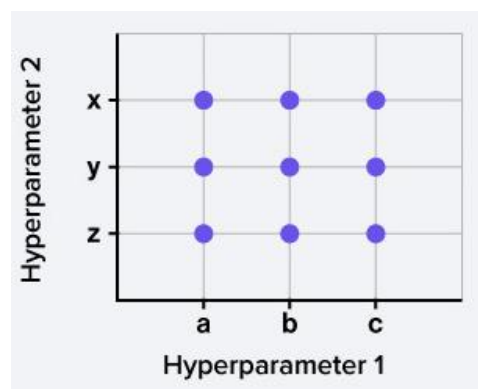


Figure 8: Grid Search

Random search samples hyperparameter combinations from specified distributions. Instead of evaluating all combinations, it tests a fixed number of random configurations. `RandomizedSearchCV` is often more efficient than grid search for large or continuous search spaces and can find strong configurations with fewer evaluations.

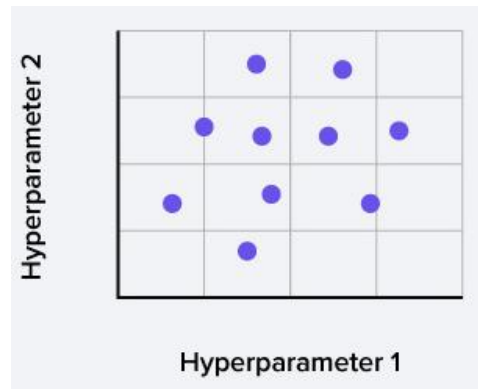


Figure 9: Random Search

Effective hyperparameter search also involves controlling the search process. This includes defining realistic parameter ranges, using early stopping to terminate unpromising runs and planning computational budgets through coarse-to-fine search strategies.

Common Pitfalls in Cross-Validation

Data leakage occurs when information from validation or test data influences model training. A common example is fitting preprocessing steps such as scaling on the full data set before splitting. To avoid leakage, data should be split first, and transformations should be learnt only on training data.

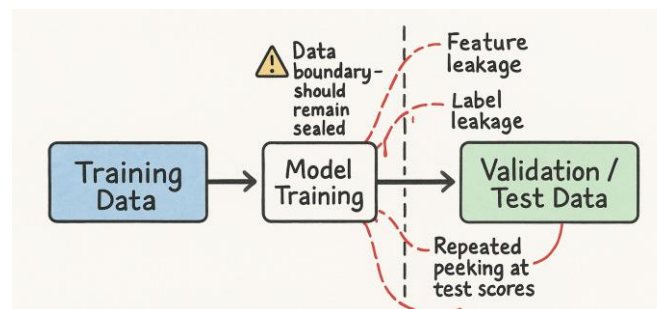


Figure 10: Leakage in CV

Target leakage arises when predictors include information derived from the target that would not be available at the time of deployment. Such features must be excluded to ensure realistic performance estimates.

Repeated reuse of a validation set for tuning can also lead to overfitting on the validation data itself. A separate test set or nested cross-validation helps mitigate this risk.

Curse of Dimensionality and Feature Selection

As the number of features increases, the feature space grows exponentially, data becomes sparse, distances become less meaningful and models require more data to learn reliable patterns. This phenomenon is called the curse of dimensionality.

High-dimensional models are more prone to overfitting and instability because sparse data makes it difficult for the model to reliably estimate relationships between variables. As dimensionality increases, each additional feature adds uncertainty. Feature selection helps address this by reducing dimensionality, removing noise and improving interpretability and computational efficiency.

Feature Selection Approaches

Feature selection methods aim to retain informative features while discarding irrelevant or redundant ones. These methods fall into three broad categories: filter methods, wrapper methods and embedded methods.

Filter methods evaluate features independently of any model using statistical measures such as correlation or mutual information. They are fast and model-agnostic, making them suitable as an initial screening step.

Correlation measures the strength and direction of linear relationships between features and the target. Pearson correlation ranges from -1 to 1 and provides a quick first-pass assessment when linearity is reasonable.

Mutual information measures how much knowing a feature reduces uncertainty about the target. It can capture linear and nonlinear dependencies and is suitable for continuous and categorical variables though it is more computationally demanding.

Several other filter-based criteria are available, including measures such as the chi-squared statistic and multivariate filtering techniques. Sometimes, correction techniques such as Bonferroni correction or false discovery rate are used to manage the chances of false discoveries.

Wrapper methods use a predictive model to evaluate subsets of features based on validation performance. They can capture feature interactions but are computationally expensive because the model is trained multiple times.

These methods evaluate feature subsets by training and validating a model. Recursive feature elimination (RFE) is a common approach in which the model is trained, features are ranked by importance, the weakest feature is removed and the process is repeated.

RFECV extends this idea by performing RFE within cross-validation. It evaluates the performance at each feature count and selects the subset that offers the best generalisation. This approach is powerful but computationally intensive.

Embedded methods perform feature selection as part of model training itself. Examples include L1-regularised models, tree-based feature importance and permutation importance. These methods balance efficiency and effectiveness by integrating selection into the learning process.

L1 regularisation encourages sparsity by shrinking some coefficients to zero. Tree-based models assign importance based on impurity reduction, naturally highlighting informative features.

Permutation importance evaluates the extent of degradation of the model performance when a feature's values are randomly shuffled. A large drop indicates that the feature is important.

Conclusion

Feature selection is prone to label or data leakage and must be performed within the training folds of cross-validation, not on the full data set. Performing selection before splitting introduces leakage and leads to overly optimistic performance estimates.

Nested cross-validation provides a robust solution using an inner loop for feature and model selection and an outer loop for unbiased performance evaluation. This approach is crucial in high-stakes applications where reliable estimates are critical.

Model selection is a structured process that combines validation, cross-validation, hyperparameter tuning and feature selection. Its goal is to choose models that generalise reliably and align with real-world objectives, not to maximise training performance. By following leakage-safe practices and aligning metrics with business goals, we can make principled and trustworthy model selection decisions.

Disclaimer: All content and material on the upGrad website is copyrighted material, either belonging to upGrad or its bona fide contributors and is purely for the dissemination of education. You are permitted to access, print, and download extracts from this site purely for your own education only and on the following basis:

- You can download this document from the website for self-use only.
- Any copies of this document, in part or full, saved to disk or to any other storage medium, may only be used for subsequent, self-viewing purposes or to print an individual extract or copy for non-commercial personal use only.
- Any further dissemination, distribution, reproduction or copying of the content of the document herein or the uploading thereof on other websites, or use of the content for any other commercial/unauthorized purposes in any way which could infringe the intellectual property rights of upGrad or its contributors, is strictly prohibited.
- No graphics, images, or photographs from any accompanying text in this document will be used separately for unauthorized purposes.
- No material in this document will be modified, adapted, or altered in any way.
- No part of this document or upGrad content may be reproduced or stored on any other website or included in any public or private electronic retrieval system or service without prior written permission from upGrad.
- Any rights not expressly granted in these terms are reserved.